EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos

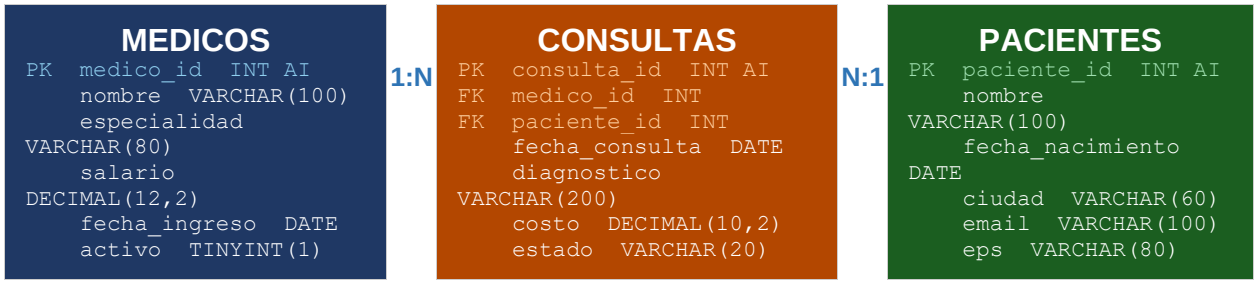
Nombre:	Juanita Castrillon		
Docente:	Tatiana Cabrera	**Fecha:**	20/04/2026

INSTRUCCIONES:

1. Cada pregunta integra múltiples temáticas SQL; lea con atención el enunciado completo.
2. Escriba sentencias con sintaxis válida para MySQL
3. Cadda estudiante solo debe resolver la pregunta asignada.
4. Se evalúa: correctitud lógica, sintaxis, eficiencia y uso de las cláusulas indicadas.
5. No se permite material de apoyo, celulares ni calculadoras.
6. **Deben entregar:**
   - Pantallazos de los resultados de ejecución
   - Script con las sentencias recuerda documentar el código.
   - Cargar el script y los pantallazos en el repositorio de trabajos en un paquete que se llame Segundo parcial Práctico.
   - Incluir en la plataforma el enlace del repositorio, el Script y el PDF de los pantallazos obtenidos de la ejecución.

CASO: SISTEMA DE GESTION HOSPITALARIA

La base de datos modela la operacion de un hospital. Contiene medicos con su especialidad y salario, pacientes con su EPS y datos personales, y consultas medicas que relacionan medicos con pacientes incluyendo diagnostico, costo y estado.



SCRIPT DDL / DML

```
-- ================================================================
-- CASO: Sistema de Gestion Hospitalaria
-- Tablas: medicos, pacientes, consultas
-- ================================================================
DROP DATABASE IF EXISTS hospital_db;
CREATE DATABASE hospital_db CHARACTER SET utf8mb4;
USE hospital_db;

CREATE TABLE medicos (
    medico_id        INT AUTO_INCREMENT PRIMARY KEY,
    nombre           VARCHAR(100) NOT NULL,
    especialidad     VARCHAR(80)  NOT NULL,
    salario          DECIMAL(12,2) NOT NULL CHECK (salario > 0),
    fecha_ingreso    DATE NOT NULL,
    activo           TINYINT(1) DEFAULT 1
```

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos

```
);

CREATE TABLE pacientes (
    paciente_id      INT AUTO_INCREMENT PRIMARY KEY,
    nombre           VARCHAR(100) NOT NULL,
    fecha_nacimiento DATE NOT NULL,
    ciudad           VARCHAR(60),
    email            VARCHAR(100) UNIQUE,
    eps              VARCHAR(80)
);

CREATE TABLE consultas (
    consulta_id      INT AUTO_INCREMENT PRIMARY KEY,
    medico_id        INT NOT NULL,
    paciente_id      INT NOT NULL,
    fecha_consulta   DATE NOT NULL,
    diagnostico      VARCHAR(200),
    costo            DECIMAL(10,2) NOT NULL CHECK (costo > 0),
    estado           VARCHAR(20) DEFAULT "programada"
        CHECK (estado IN ("programada","realizada","cancelada")),
    FOREIGN KEY (medico_id)   REFERENCES medicos(medico_id),
    FOREIGN KEY (paciente_id) REFERENCES pacientes(paciente_id)
);

-- ================================================================
-- DML: DATOS DE PRUEBA
-- ================================================================
INSERT INTO medicos VALUES
 (1,"Dra. Laura Rios","Cardiologia",8500000.00,"2018-03-10",1),
 (2,"Dr. Carlos Mesa","Neurologia",9200000.00,"2016-07-22",1),
 (3,"Dra. Sofia Vega","Pediatria",7800000.00,"2020-01-15",1),
 (4,"Dr. Andres Gil","Ortopedia",8100000.00,"2019-06-01",1),
 (5,"Dra. Paula Mora","Cardiologia",8700000.00,"2017-09-30",1),
 (6,"Dr. Ivan Cruz","Dermatologia",7500000.00,"2021-04-12",0),
 (7,"Dra. Marta Leon","Neurologia",9500000.00,"2015-11-05",1),
 (8,"Dr. Felipe Ossa","Pediatria",7600000.00,"2022-02-28",1);

INSERT INTO pacientes VALUES
 (1,"Juan Perez","1985-04-12","Bogota","juan@mail.com","Sura"),
 (2,"Ana Gomez","1992-08-25","Medellin","ana@mail.com","Compensar"),
 (3,"Luis Vargas","1978-12-03","Cali","luis@mail.com","Sura"),
 (4,"Maria Diaz","2001-06-17","Bogota","maria@mail.com","Famisanar"),
 (5,"Carlos Ruiz","1965-01-30","Barranquilla","carlos@mail.com","Compensar"),
 (6,"Lucia Herrera","1990-09-08","Bogota","lucia@mail.com","Sura"),
 (7,"Pedro Soto","2005-03-22","Cali","pedro@mail.com","Famisanar"),
 (8,"Valeria Torres","1998-11-14","Medellin","valeria@mail.com","Compensar");

INSERT INTO consultas VALUES
 (1,1,1,"2024-01-10","Hipertension leve",150000,"realizada"),
 (2,1,3,"2024-01-22","Control cardiaco",150000,"realizada"),
 (3,2,2,"2024-02-05","Cefalea cronica",200000,"realizada"),
 (4,2,5,"2024-02-18","Migraña",200000,"cancelada"),
 (5,3,4,"2024-03-01","Control crecimiento",90000,"realizada"),
 (6,3,7,"2024-03-14","Fiebre alta",90000,"realizada"),
 (7,4,6,"2024-04-02","Fractura muñeca",250000,"realizada"),
```

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos

```
(8,4,1,"2024-04-15","Dolor rodilla",250000,"programada"),
(9,5,8,"2024-05-03","Arritmia",180000,"realizada"),
(10,5,2,"2024-05-20","Ecocardiograma",180000,"realizada"),
(11,6,3,"2024-05-28","Dermatitis",120000,"cancelada"),
(12,7,5,"2024-06-10","Epilepsia control",220000,"realizada"),
(13,7,6,"2024-06-22","Resonancia",220000,"programada"),
(14,8,4,"2024-07-01","Vacunacion",60000,"realizada"),
(15,1,8,"2024-07-15","Hipertension severa",180000,"realizada"),
(16,3,2,"2024-07-28","Seguimiento",90000,"programada");
```

PREGUNTAS — PARCIAL No. 2

No.	Temas	Enunciado
1	DDL	Sobre la base de datos hospital_db ya creada, realice los siguientes cambios estructurales: (a) Agregue a la tabla pacientes una columna telefono VARCHAR(15) que permita valores nulos. (b) Agregue a la tabla medicos una columna consultorio VARCHAR(10) NOT NULL DEFAULT "A1". (c) Cree un indice unico sobre el campo email de la tabla pacientes (si no existe ya como UNIQUE en la definicion). (d) Elimine la columna activo de la tabla medicos. Clausulas requeridas: ALTER TABLE ADD COLUMN, CREATE INDEX, ALTER TABLE DROP COLUMN
2	DML	Ejecute las siguientes operaciones de manipulacion de datos: (a) Inserte un nuevo medico: nombre=Dr. Jorge Pino, especialidad=Ginecologia, salario=8300000, fecha_ingreso=2024-01-10, activo=1. (b) Inserte un nuevo paciente: nombre=Daniela Cano, fecha_nacimiento=1995-07-20, ciudad=Pereira, email=daniela@mail.com, eps=Sura. (c) Actualice el salario de todos los medicos de la especialidad Pediatria incrementandolo un 12%. (d) Elimine todas las consultas con estado cancelada cuya fecha_consulta sea anterior a 2024-03-01. Clausulas requeridas: INSERT INTO, UPDATE con WHERE, DELETE con WHERE
3	DDL DML	Cree la tabla historial_salarios (historial_id PK AUTO_INCREMENT, medico_id FK referencias medicos, salario_anterior DECIMAL(12,2), salario_nuevo DECIMAL(12,2), fecha_cambio DATETIME DEFAULT NOW(), motivo VARCHAR(150)). Agregue ademas la restriccion CHECK en consultas para que el costo sea mayor a 0 (si no existe). Luego inserte dos registros en historial_salarios reflejando cambios de salario para los medicos con id 1 y 3, con los valores que ya existen en la tabla medicos como salario_nuevo. Clausulas requeridas: CREATE TABLE con FK y CHECK, ALTER TABLE ADD CONSTRAINT, INSERT multiple
4	DML Multi	Realice en secuencia las siguientes operaciones: (a) Inserte una nueva consulta para el medico_id=3 y el paciente_id=5, con fecha=CURDATE(), diagnostico=Control pediarico, costo=90000, estado=programada. (b) Actualice el estado de esa consulta a realizada. (c) Consulte con un JOIN (medicos, consultas, pacientes) el nombre del medico, nombre del paciente, fecha_consulta y costo de todas las consultas realizadas por la Dra. Sofia Vega. Clausulas requeridas: INSERT, UPDATE, SELECT con JOIN tres tablas, WHERE por nombre de medico

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos

5	Multi Sub	Muestre el nombre del medico, especialidad, nombre del paciente y costo de todas las consultas realizadas cuyo costo sea mayor al costo promedio de todas las consultas realizadas (use subconsulta escalar en el WHERE). Ordene los resultados por costo descendente. Clausulas requeridas: JOIN tres tablas, WHERE costo > (SELECT AVG(costo) ...), ORDER BY DESC
6	Multi Vista	Cree la vista vista_consultas_realizadas que muestre: consulta_id, nombre del medico, especialidad, nombre del paciente, eps del paciente, fecha_consulta, diagnostico y costo, para todas las consultas con estado realizada. Luego consulte la vista para obtener el total de consultas y la suma de costos agrupados por especialidad. Clausulas requeridas: CREATE VIEW con JOIN tres tablas y WHERE, SELECT FROM vista con GROUP BY y SUM
7	Sub Vista	Cree la vista vista_pacientes_frecuentes que liste el paciente_id, nombre, ciudad y eps de los pacientes que tienen mas de una consulta registrada (en cualquier estado). Use una subconsulta en el WHERE o un HAVING. Luego consulte la vista para mostrar cuantos pacientes frecuentes hay por ciudad. Clausulas requeridas: CREATE VIEW con subconsulta COUNT > 1 o HAVING, SELECT FROM vista GROUP BY ciudad
8	Func Multi	Cree la funcion fn_edad_paciente(p_paciente_id INT) que retorne la edad en años del paciente usando TIMESTAMPDIFF(YEAR, fecha_nacimiento, CURDATE()). Luego use esa funcion en un SELECT sobre la tabla pacientes con JOIN a consultas para mostrar nombre del paciente, eps, edad (usando la funcion) y total de consultas realizadas, ordenado por edad descendente. Clausulas requeridas: CREATE FUNCTION con SELECT y TIMESTAMPDIFF, SELECT con JOIN usando la funcion, GROUP BY, ORDER BY
9	Func Sub	Cree la funcion fn_costo_total_paciente(p_paciente_id INT) que retorne la suma de costos de todas las consultas realizadas del paciente (solo estado=realizada). Luego escriba una consulta sobre la tabla pacientes que muestre nombre, ciudad, eps y costo_total (usando la funcion), filtrando solo los pacientes cuyo costo_total supere el promedio de costo_total de todos los pacientes (use subconsulta con la misma funcion o calculo directo). Clausulas requeridas: CREATE FUNCTION con SELECT SUM, SELECT usando funcion en columna y en WHERE con subconsulta
10	Proc DML Multi	Cree el procedimiento sp_registrar_consulta(p_medico_id INT, p_paciente_id INT, p_fecha DATE, p_diagnostico VARCHAR(200), p_costo DECIMAL(10,2)) que: (a) Verifique que el medico exista y este activo (activo=1); si no, lance un error con SIGNAL SQLSTATE. (b) Verifique que el paciente exista; si no, lance error. (c) Inserte la consulta con estado programada. (d) Retorne con un SELECT JOIN el nombre del medico, nombre del paciente y los datos de la consulta recien insertada. Clausulas requeridas: CREATE PROCEDURE, SELECT INTO var para validar, IF/SIGNAL SQLSTATE, INSERT, SELECT con JOIN
11	Proc Vista DML	Cree la vista vista_agenda_pendiente que muestre: consulta_id, nombre del medico, especialidad, nombre del paciente, fecha_consulta y dias_espera (DATEDIFF entre fecha_consulta y CURDATE()) para todas las consultas en estado programada. Luego cree el procedimiento sp_cancelar_consultas_vencidas(p_dias_limite INT) que: (a) Consulte la vista para identificar consultas cuyo dias_espera sea mayor a p_dias_limite (es decir, con fecha pasada). (b) Actualice esas consultas a estado cancelada. (c) Retorne el numero de filas afectadas con ROW_COUNT(). Clausulas requeridas: CREATE VIEW con JOIN y DATEDIFF, CREATE PROCEDURE con UPDATE usando condicion de la vista, ROW_COUNT()

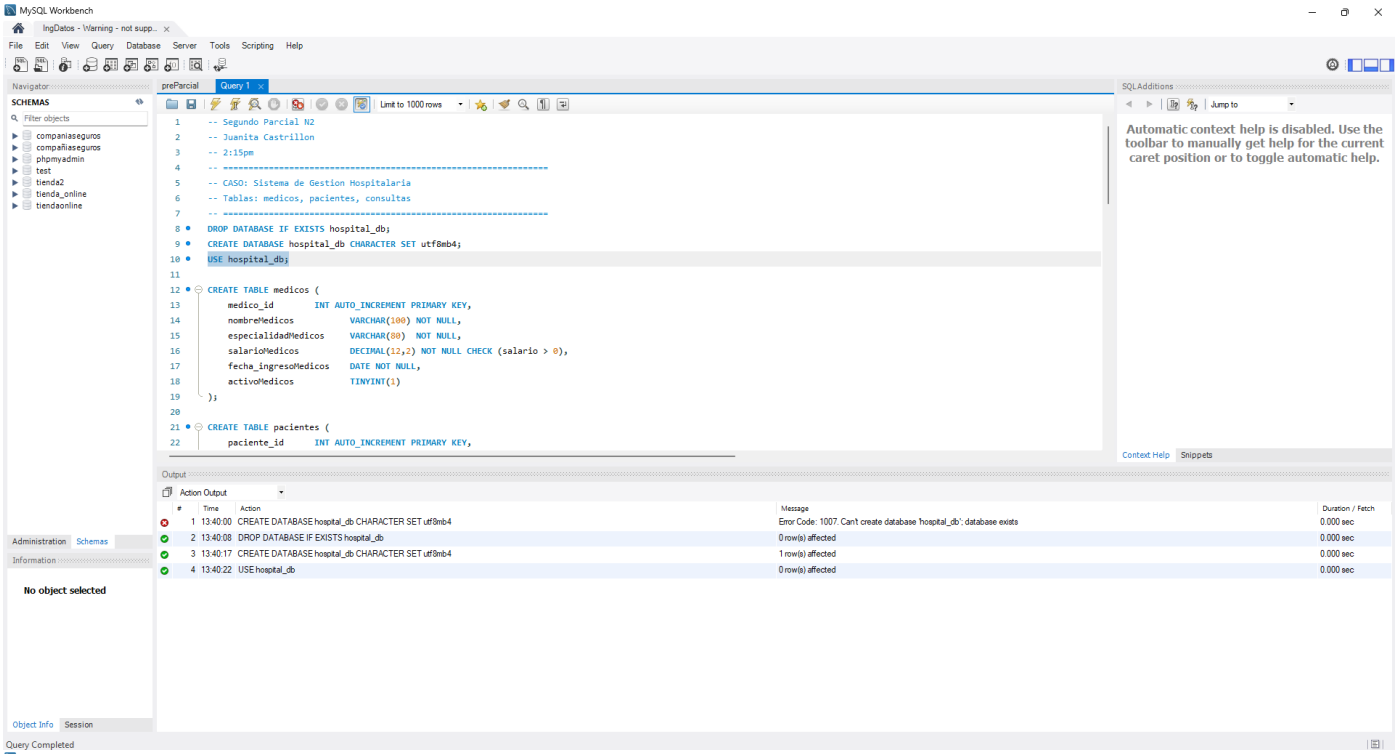
EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

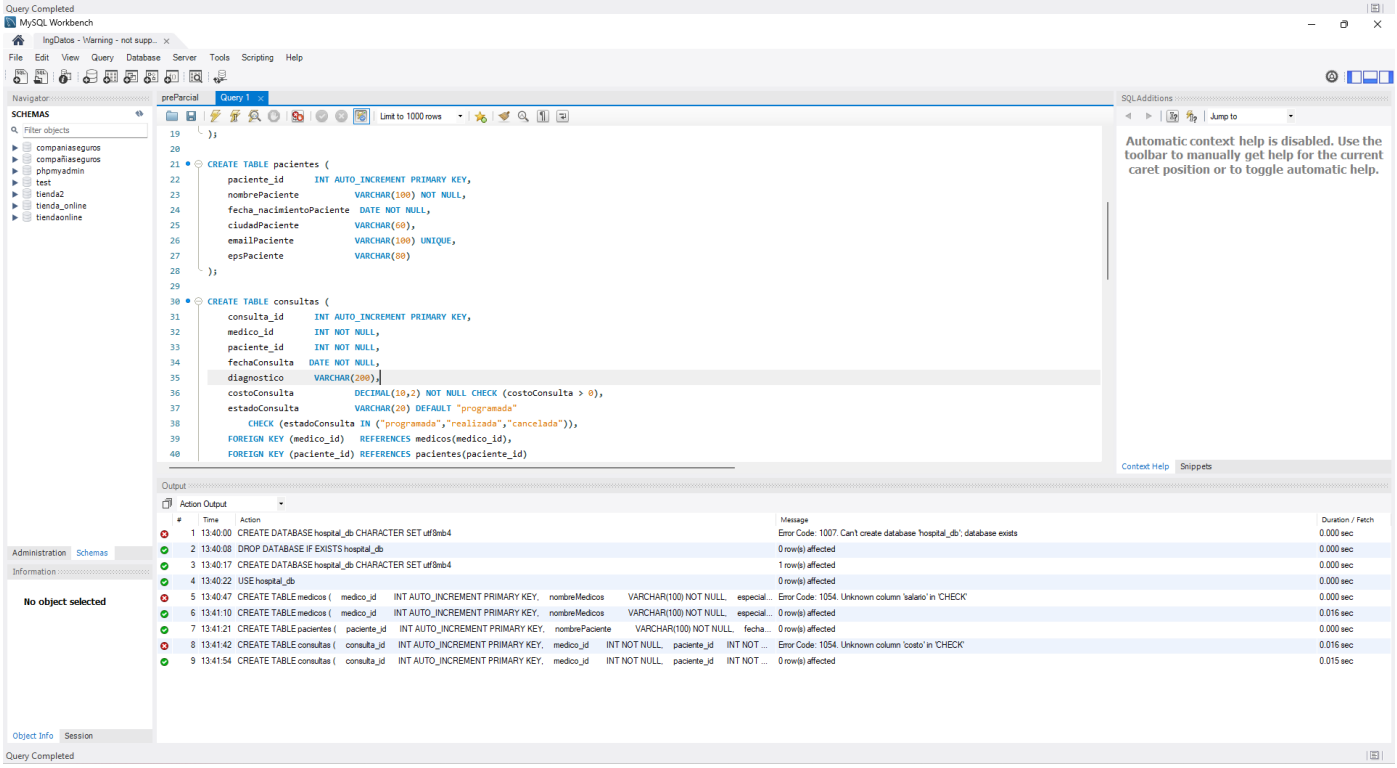
Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos

12	DDL Func Sub	Agregue la columna nivel_atencion VARCHAR(20) DEFAULT NULL a la tabla consultas. Cree la funcion fn_nivel_atencion(p_costo DECIMAL(10,2)) que retorne: BASICO si costo <= 100000, INTERMEDIO si entre 100001 y 200000, ESPECIALIZADO si > 200000. Luego actualice la columna nivel_atencion de todas las consultas usando la funcion en un UPDATE. Finalmente, escriba una consulta que muestre cuantas consultas hay por nivel_atencion (excluyendo las canceladas), ordenado de mayor a menor. Clausulas requeridas: ALTER TABLE ADD COLUMN, CREATE FUNCTION con CASE, UPDATE SET col=funcion(), SELECT GROUP BY con WHERE y ORDER BY
13	DML Proc Multi	Cree el procedimiento sp_reasignar_consulta(p_consulta_id INT, p_nuevo_medico_id INT) que: (a) Verifique que la consulta exista y este en estado programada (si no, error). (b) Verifique que el nuevo medico exista, este activo y tenga la misma especialidad que el medico actual asignado (si no coincide, error). (c) Inserte un registro en historial_salarios usando los datos del medico original como referencia (registre el cambio simbolico). (d) Actualice el medico_id de la consulta. (e) Retorne con SELECT JOIN el detalle actualizado de la consulta. Clausulas requeridas: CREATE PROCEDURE, multiples SELECT INTO para validaciones, IF/SIGNAL, INSERT, UPDATE, SELECT con JOIN
14	Multi Func Vista	Cree la funcion fn_clasificar_medico(p_medico_id INT) que retorne: EXPERTO si el medico tiene 5 o mas consultas realizadas, INTERMEDIO entre 2 y 4, y JUNIOR si tiene 0 o 1. Cree la vista vista_rendimiento_medicos que muestre: medico_id, nombre, especialidad, salario, total_consultas_realizadas, total_ingresos_generados (SUM de costos realizadas) y clasificacion (usando la funcion). Consulte la vista para listar solo los medicos EXPERTO ordenados por total_ingresos_generados descendente. Clausulas requeridas: CREATE FUNCTION con SELECT COUNT, CREATE VIEW con LEFT JOIN y funcion en columna, SELECT FROM vista con WHERE y ORDER BY
15	Sub Multi Proc	Cree el procedimiento sp_informe_especialidad(p_especialidad VARCHAR(80)) que: (a) Valide que exista al menos un medico activo con esa especialidad (si no, error). (b) Retorne con un SELECT el resumen de esa especialidad: total_medicos_activos, promedio_salario, total_consultas_realizadas, ingreso_total_generado, paciente_mas_frecuente (nombre del paciente con mas consultas en esa especialidad, usando subconsulta). (c) Adicionalmente liste con otro SELECT los nombres de los pacientes que han tenido consultas con esa especialidad pero que TAMBIEN hayan tenido consultas canceladas en cualquier especialidad (use interseccion con subconsultas IN). Clausulas requeridas: CREATE PROCEDURE, multiples SELECT con JOIN, subconsulta escalar para paciente_mas_frecuente, subconsulta con IN doble
16	Sub Multi Vista Func	Integre todos los elementos aprendidos: (a) Cree la funcion fn_ingreso_medico_mes(p_medico_id INT, p_mes INT, p_anio INT) que retorne la suma de costos de consultas realizadas por ese medico en el mes y anio indicados. (b) Cree la vista vista_medicos_top_mes que muestre, para el mes de julio de 2024, el nombre del medico, especialidad, ingresos_julio (usando la funcion) y lo compare contra el promedio de ingresos de julio de todos los medicos (subconsulta escalar); incluya una columna supera_promedio que sea SI o NO. (c) Consulte la vista junto con un JOIN a consultas para listar el diagnostico y paciente de cada consulta realizada en julio 2024 solo de los medicos que superan el promedio. Ordene por ingresos_julio descendente. Clausulas requeridas: CREATE FUNCTION con WHERE MONTH/YEAR, CREATE VIEW con subconsulta escalar y CASE, SELECT JOIN sobre vista con WHERE supera_promedio=SI

EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

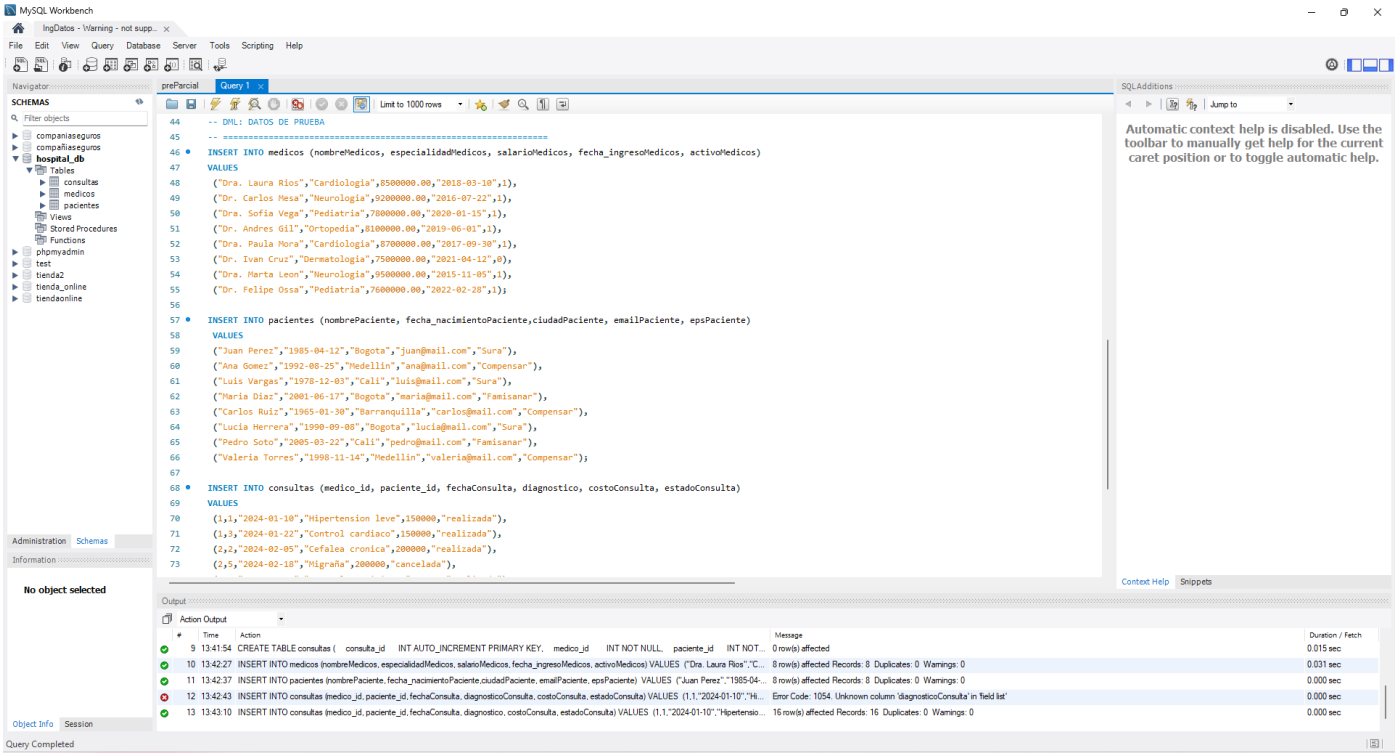
Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos

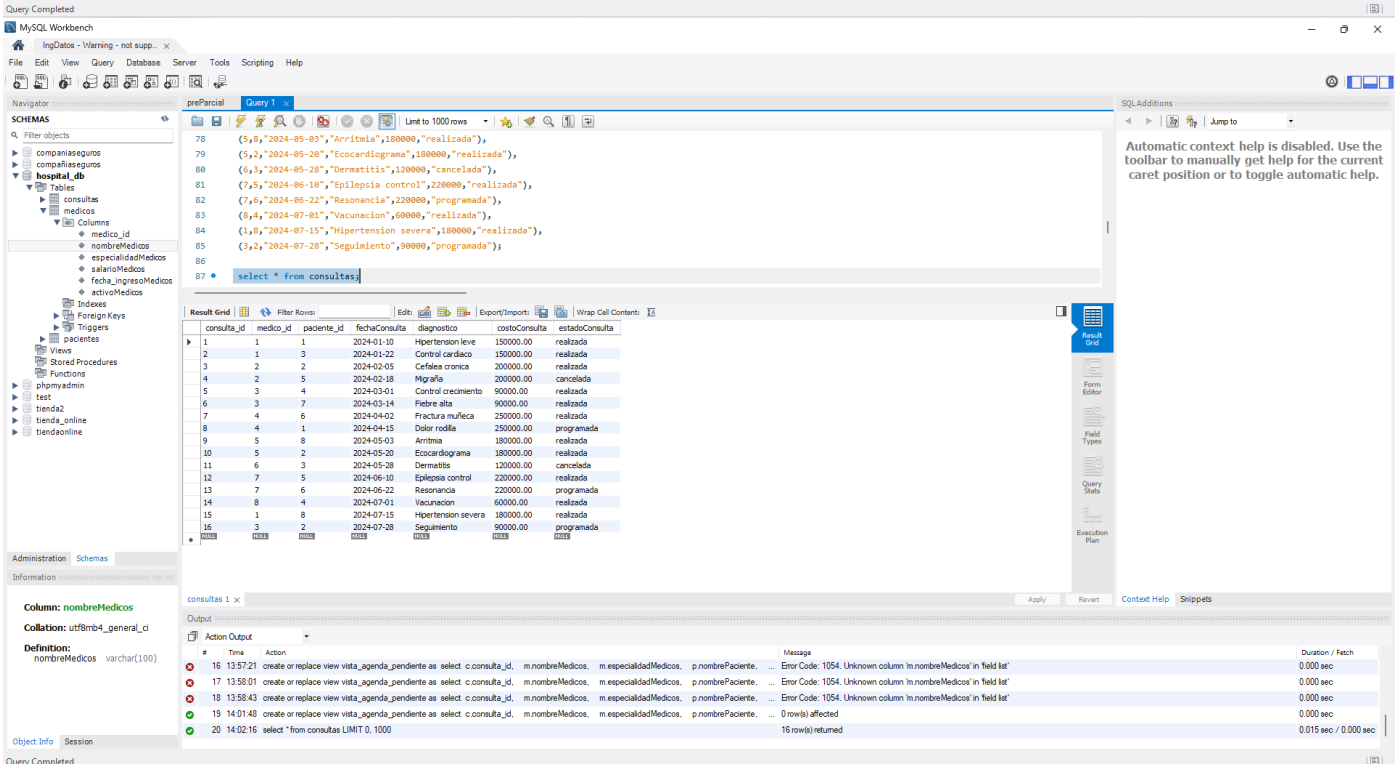




EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

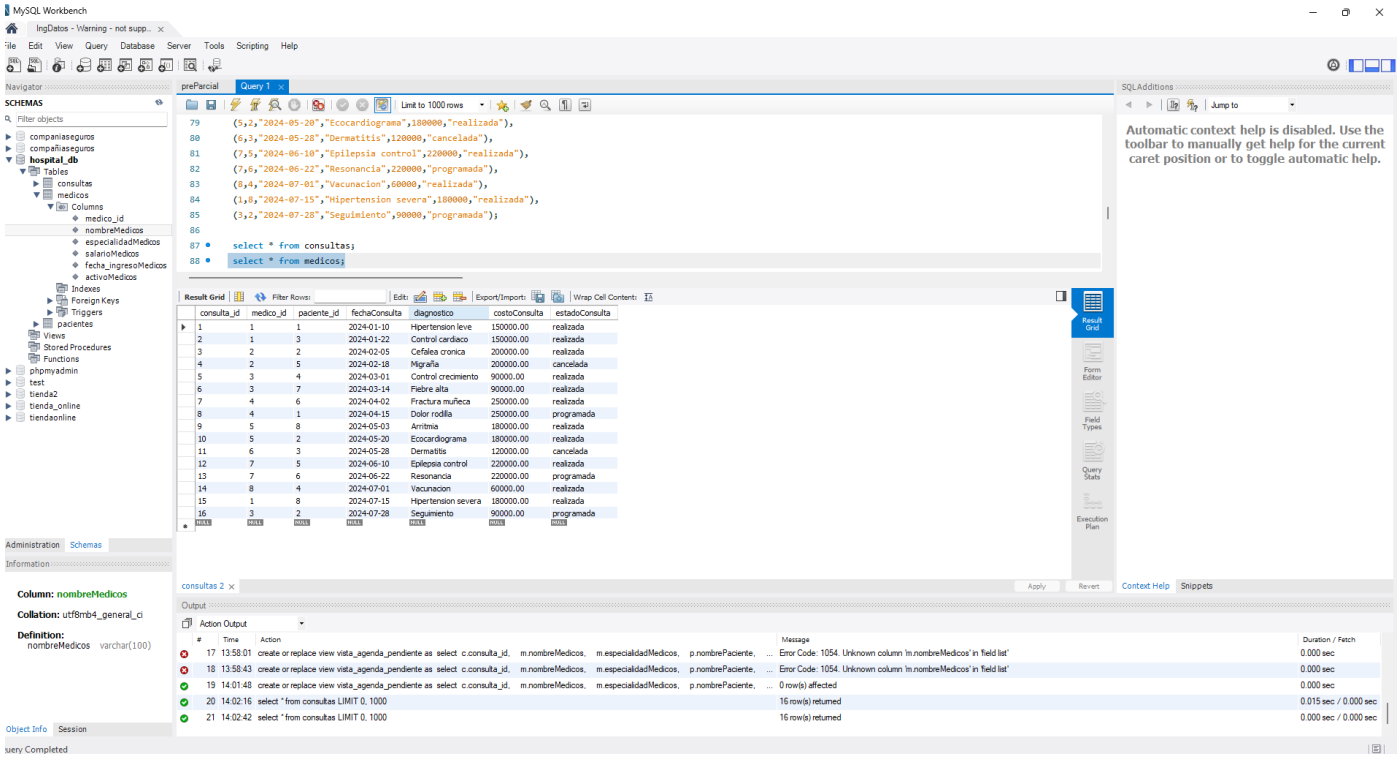
Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos

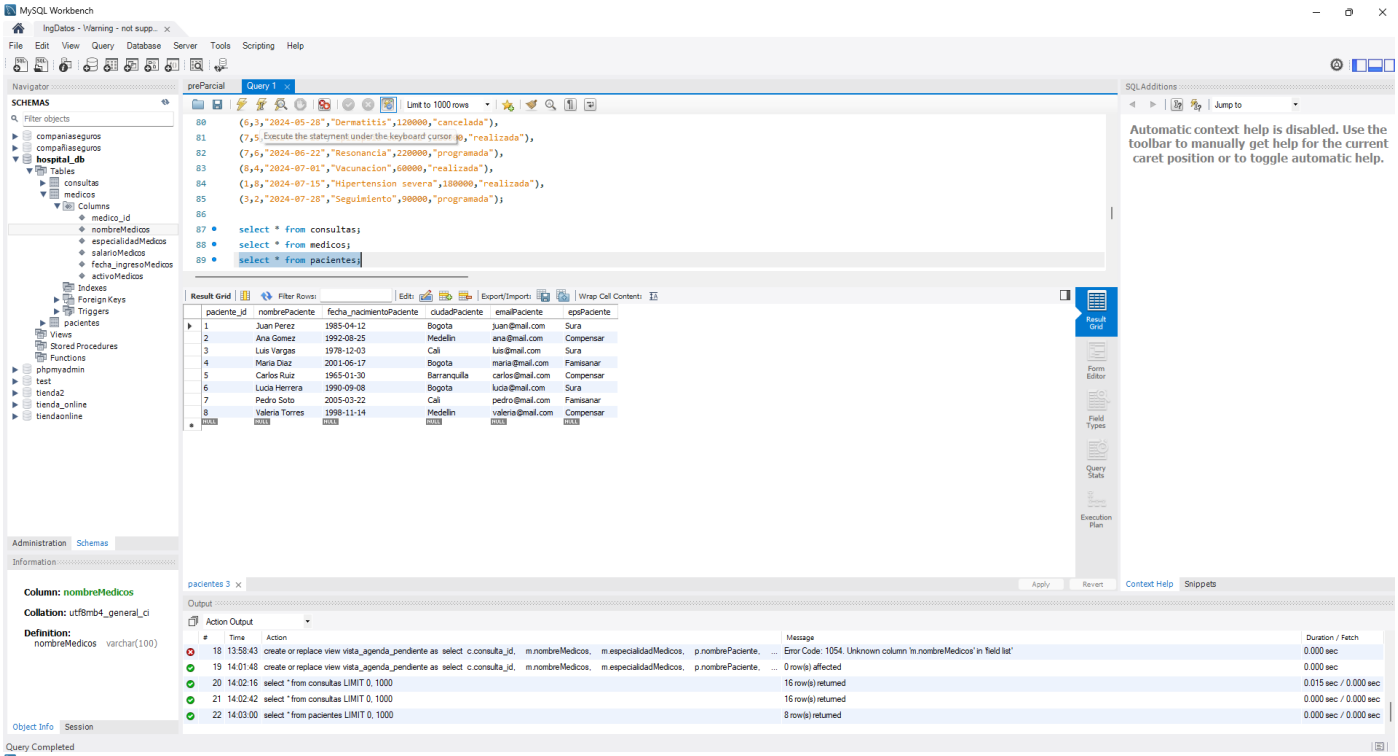
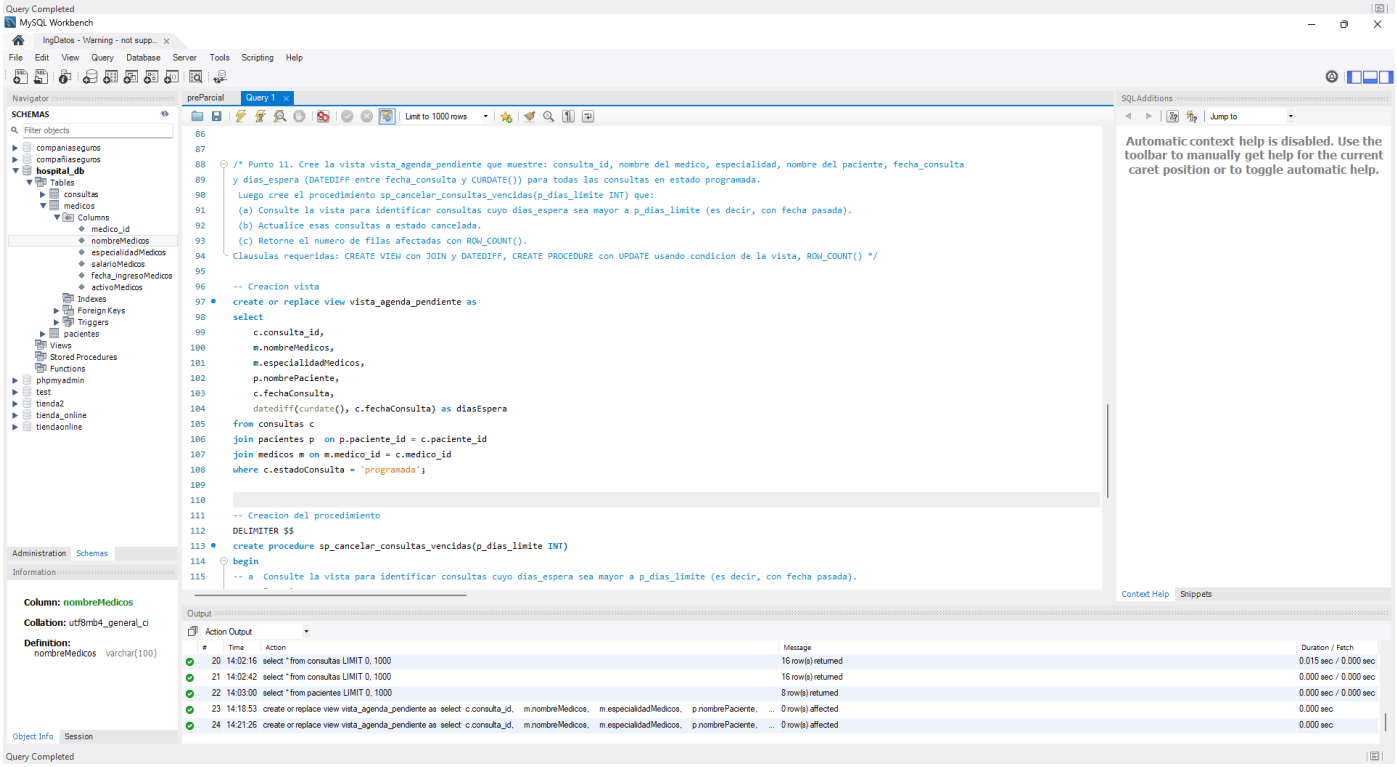




EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos



EVALUACION PARCIAL — LENGUAJE SQL CON MYSQL

Ingeniería de Datos • PARCIAL No. 2 Duración: 90 minutos